

1.4 Ý Nghĩa Phần Cứng và Giới Hạn Thời Gian Thực

Mục 1.3 đã khảo sát chi phí tính toán lý thuyết của từng toán tử biến đổi phổ trên một khung dữ liệu độc lập. Khi chuyển các toán tử đó sang môi trường phần cứng biên xử lý tín hiệu theo dòng liên tục, những câu hỏi cốt lõi về thời gian thực, dung lượng lưu trữ nội vi và mô hình hàng đợi bắt đầu xuất hiện.

► MỤC TIÊU HỌC TẬP

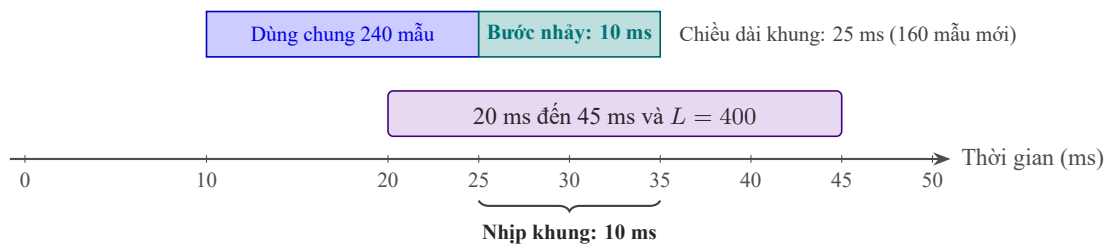
1. Các toán tử biến đổi phổ trong Mục 1.3 có chi phí tính toán cho mỗi khung rời rạc, nhưng khi nào kết quả đầu tiên của chuỗi xử lý âm thanh có thể xuất hiện trên mạch vật lý?
2. 400 mẫu tín hiệu của một khung cửa sổ được lưu trữ ở đâu trong tài nguyên FPGA và chiếm bao nhiêu dung lượng của một khối Block RAM đơn lẻ?
3. Ma trận ngân hàng bộ lọc Mel kích thước 80×257 thực tế tiêu tốn bao nhiêu dung lượng lưu trữ trọng số và bao nhiêu phép tính số học mỗi khung?
4. Việc di chuyển tiền xử lý âm thanh lên phần cứng chuyên dụng có phải là một thắng lợi về thông lượng tính toán so với Edge GPU hay không?
5. Nếu khối lượng tính toán số học rất nhỏ, điều gì vẫn có thể làm trễ hạn chót 10 ms của bước nhảy khung âm thanh?
6. Hiện tượng gì xảy ra ở ranh giới 1 giây âm thanh đối với các mẫu dữ liệu cuối cùng, và việc gom cụm khung (batching) tạo ra chi phí trễ thời gian như thế nào?

1. Ranh Giới Thời Gian Khung Đầu Tiên và Nhịp Bước Nhảy

Câu hỏi giai đoạn trước không thể trả lời: Các toán tử trong Mục 1.3 có chi phí tính toán xác định cho mỗi khung âm thanh rời rạc, nhưng khi nào kết quả đầu tiên thực sự có thể tồn tại trên mạch vật lý?

Khung 0: Chiều dài 25 ms ($L = 400$ mẫu)

Mẫu thứ 400 chưa đến trước thời điểm 25 ms



Hình 1.4a: **Bản chất:** Tiến trình thời gian của các khung âm thanh nối tiếp và cơ chế gởi đầu tại tần số lấy mẫu $f_s = 16.000$ Hz. **Điểm cần quan sát:** Khung 0 đòi hỏi đúng 25 ms để thu thập đủ 400 mẫu; Khung 1 tái sử dụng 240 mẫu cũ và nhận thêm 160 mẫu mới sau bước nhảy 10 ms; Khung 2 hoàn tất tại 45 ms; nhịp xuất khung sau đó đều đặn rơi vào mỗi chu kỳ 10 ms. **Ý nghĩa:** Ranh giới vật lý 25 ms là sản phẩm tự nhiên không thể rút ngắn bằng việc tăng tần số xung nhịp; sau khung đầu tiên, toàn bộ chuỗi toán tử phải hoàn tất trong khoảng thời gian khắt khe 10 ms.

Với tần số lấy mẫu $f_s = 16.000$ Hz và chiều dài khung $L = 400$ mẫu, thời gian cần thiết để thu thập đủ một khung dữ liệu hoàn chỉnh là:

$$\frac{L}{f_s} = \frac{400}{16000} = 0,025 \text{ s} = 25 \text{ ms}$$

Bước nhảy giữa hai khung liên tiếp là $H = 160$ mẫu, tương ứng với khoảng thời gian bước nhảy:

$$\frac{H}{f_s} = \frac{160}{16000} = 0,010 \text{ s} = 10 \text{ ms}$$

Hai khung kế tiếp chia sẻ một lượng mẫu âm thanh gởi đầu:

$$L - H = 400 - 160 = 240 \text{ mẫu}$$

Một xung nhịp xử lý nhanh hơn không thể làm dịch chuyển ranh giới vật lý 25 ms này, bởi mẫu thứ 400 đơn giản là chưa xuất hiện từ bộ chuyển đổi tín hiệu trước thời điểm đó. Sau khi khung đầu tiên hoàn tất, mỗi khung tiếp theo phải được bàn giao sau đúng mỗi chu kỳ 10 ms. Khoảng thời gian 10 ms đó phải chứa trọn vẹn toàn bộ chuỗi xử lý: áp cửa sổ Hamming, biến đổi FFT, tính phổ công suất, tổng năng lượng ngân hàng bộ lọc Mel, nén logarit, và mô hình âm học.

2. Nơi Lưu Trữ 400 Mẫu Âm Thanh: Khối Block RAM Nội Vi

Câu hỏi giai đoạn trước không thể trả lời: 400 mẫu âm thanh của khung quan sát nằm ở đâu trong kiến trúc phần cứng khi dữ liệu liên tục đổ vào?

Xét đúng một ngăn chứa vật lý trên FPGA: một khối Block RAM (BRAM) chuẩn 36 kbit có dung lượng lưu trữ tính theo byte là:

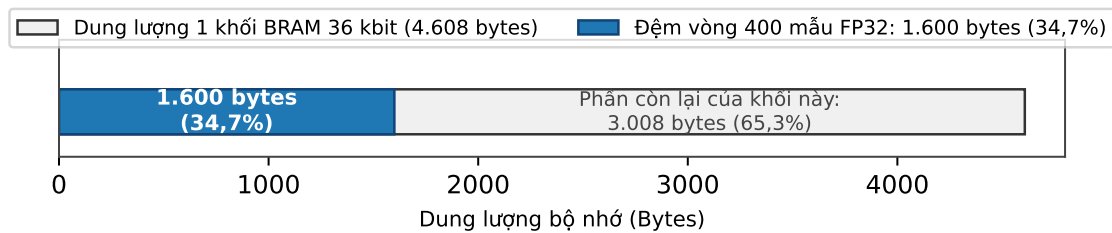
$$\frac{36 \times 1024}{8} = 4608 \text{ byte}$$

Bộ đệm vòng (ring buffer) dùng để duy trì cửa sổ trượt 400 mẫu số thực dấu phẩy động 32-bit (FP32) cần:

$$400 \times 4 = 1600 \text{ byte}$$

Tỉ lệ chiếm dụng của bộ đệm vòng so với sức chứa của đúng một khối BRAM đơn lẻ là:

$$\frac{1600}{4608} \approx 34,7\%$$



Hình 1.4b: **Bản chất:** Tương quan dung lượng giữa bộ đệm vòng 400 mẫu tín hiệu và sức chứa của một khối Block RAM 36 kbit đơn lẻ. **Điểm cần quan sát:** Bộ đệm vòng 1.600 byte chiếm 34,7% dung lượng của khối BRAM 4.608 byte này; phần còn lại của khối là 3.008 byte. **Ý nghĩa:** 3.008 byte còn lại trong khối này không đủ sức chứa bằng trọng số 3.328 byte của ngân hàng Mel nếu đặt chung ($1600 + 3328 = 4928 > 4608$), đòi hỏi các khối BRAM độc lập trong 144 khối của chip.

Tỉ lệ phần trăm 34,7% này là số byte của bộ đệm chia cho dung lượng danh định in trên tài liệu nhà sản xuất (AMD DS890 v4.10, Bảng 23, thiết bị XCK26). Đây là phép so sánh dung lượng lý thuyết thuần túy, không phải là kết quả đo đếm của một bản thiết kế đã được sắp ráp (placed design) trên chip. Dòng số liệu công bố “5.1 Mb” của nhà sản xuất cho 144 khối BRAM trên thiết bị là giá trị làm tròn của phép tính:

$$144 \times 36 \text{ kbit} = 5184 \text{ kbit}$$

Phép nhân số học từ các thông số danh định trên datasheet cho thấy 144 khối BRAM tương ứng với tổng sức chứa lý thuyết:

$$144 \times 4608 \text{ byte} = 663552 \text{ byte} = 648 \text{ KiB}$$

Số liệu datasheet ghi nhận 5184 kbit tương đương 648 KiB bộ nhớ BRAM.

3. Ngân Hàng Bộ Lọc Mel Thực Tế: Ma Trận Dày và Cấu Trúc Thừa

Câu hỏi giai đoạn trước không thể trả lời: Ngân hàng bộ lọc Mel từ Mục 1.3 thực sự lưu trữ bao nhiêu byte và thực hiện bao nhiêu phép tính khi triển khai trên phần cứng?

Nếu triển khai theo kỳ vọng ma trận dày (dense matrix) gồm 80×257 số thực dấu phẩy động 32-bit, dung lượng lưu trữ bằng trọng số là:

$$80 \times 257 \times 4 = 82240 \text{ byte}$$

và mỗi khung âm thanh đòi hỏi:

$$80 \times 257 = 20560 \text{ tích mỗi khung}$$

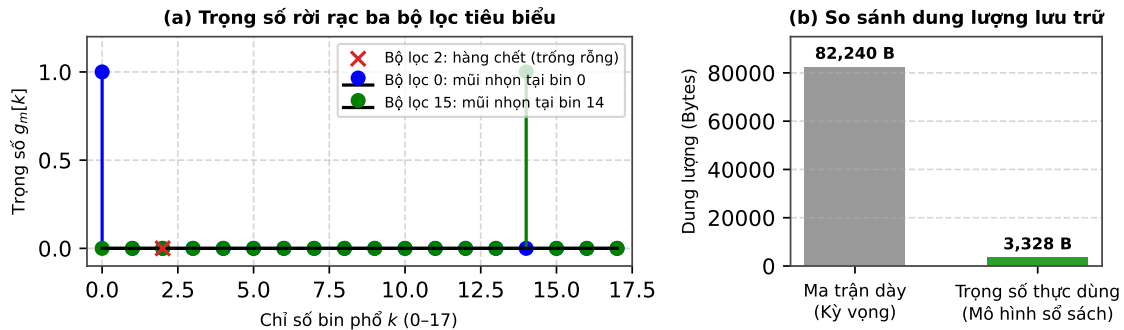
Tuy nhiên, kiểm tra ma trận do hàm `create_mel_filterbank` xây dựng cho thấy một cấu trúc thừa đặc thù:

- **Hàng chết (toàn số 0):** Có đúng 1 hàng chết (Bộ lọc 2). Kênh này luôn bằng 0, không cần lưu trữ trọng số, không tốn phép nhân và không tốn phép cộng (0 byte, 0 phép nhân, 0 phép cộng).
- **Hàng đơn điểm (one-tap) có trọng số duy nhất bằng 1:** Có 18 hàng thuộc loại này. Năng lượng đầu ra của dải băng chính công suất phổ tại bin tương ứng ($E_m = P_k$). Phần cứng chỉ cần lưu 1 chỉ số bin dạng số nguyên 32-bit (int32, 4 byte) thay vì lưu trọng số số thực, đồng thời tiêu tốn 0 phép nhân và 0 phép cộng.
- **Hàng đa điểm (multi-bin) có k trọng số dương:** Có 61 hàng thuộc loại này. Mỗi trọng số dương cần lưu 4 byte float32 cho giá trị trọng số cùng 4 byte int32 cho chỉ số bin tương ứng ($k \times 8$ byte). Mỗi hàng này đòi hỏi k phép nhân và $k - 1$ phép cộng.

Tổng dung lượng bộ nhớ thực dùng (`used_weight_bytes`) theo mô hình sổ sách của bài học là tổng dung lượng của các phần tử thực tế đó:

$$0 \text{ byte (hàng chết)} + 18 \times 4 \text{ byte (hàng đơn điểm)} + 407 \times 8 \text{ byte (hàng đa điểm)} = 72 + 3256 = 3328 \text{ byte}$$

So với kỳ vọng ma trận dày 82240 byte, mô hình lưu trữ thực dùng ghi nhận 3328 byte. Cả hai con số đều được tính toán theo quy chuẩn số thực FP32. Tổng khối lượng tính toán thực tế mỗi khung ghi nhận từ lab là `multiply_per_frame` = 407 phép nhân và `add_per_frame` = 346 phép cộng.



Hình 1.4c: **Bản chất:** Cấu trúc phân bố trọng số của ngân hàng 80 bộ lọc Mel và đối chiếu dung lượng lưu trữ sổ sách. **Điểm cần quan sát:** Bộ lọc 0 và 15 chỉ gồm một xung đơn vị đơn lẻ; Bộ lọc 2 hoàn toàn rỗng; mô hình lưu trữ thực dùng ghi nhận 3.328 byte so với 82.240 byte của ma trận dày kỳ vọng. **Ý nghĩa:** Cấu trúc thừa giúp loại bỏ phần lớn gánh nặng tính toán và thu nhỏ bảng tra trọng số, dù 3.328 byte chiếm hơn 72% dung lượng của một khối BRAM đơn lẻ.

► NHÌN THẤY VẬT LÝ

Bảng trọng số 3.328 byte chiếm tới 72,2% dung lượng của một khối BRAM 4.608 byte (do đó không thể nhét chung vào phần 3.008 byte còn lại của khối đệm vòng vì $1600 + 3328 = 4928 > 4608$). Tuy nhiên, con số 3.328 byte này rất nhỏ bé khi so với tổng dung lượng 663.552 byte của toàn bộ 144 khối BRAM trên chip. Phép chia dung lượng này là một mô hình tính toán số học trên sổ sách, không phải là một báo cáo tổng hợp mạch vật lý (synthesis report), bởi việc kết nối logic, thanh ghi trung gian và mạch tạo địa chỉ thực tế đòi hỏi các tài nguyên bổ sung trên phần cứng.

4. Đối Chiếu Thông Lượng Tính Toán: Tương Quan Quy Mô

Câu hỏi giai đoạn trước không thể trả lời: Việc di chuyển khối tiền xử lý âm thanh lên phần cứng chuyên dụng có mang lại thắng lợi về mặt thông lượng tính toán (throughput) hay không?

Với nhịp bước nhảy 10 ms, hệ thống xử lý đúng 100 khung mỗi giây. Thông lượng phép nhân thực tế của ma trận Mel là:

$$\text{multiply_per_second} = 407 \times 100 = 40700 \text{ phép nhân/giây}$$

Các thông số dẫn chiếu từ tài liệu công bố của nhà sản xuất (không phải số đo do lab này tự thực hiện):

- NVIDIA Jetson Orin Nano Super công bố định mức cực đại 67 sparse INT8 TOPS tại trần công suất 25 W.
- Độ thưa có cấu trúc (structured sparsity 2:4) nhân đôi thông lượng lý thuyết của nhân Tensor Core, do đó giá trị $67/2 = 33,5$ dense INT8 TOPS là một phép tính số học trong cuốn sách này, không phải là một thông số công bố độc lập thứ hai.

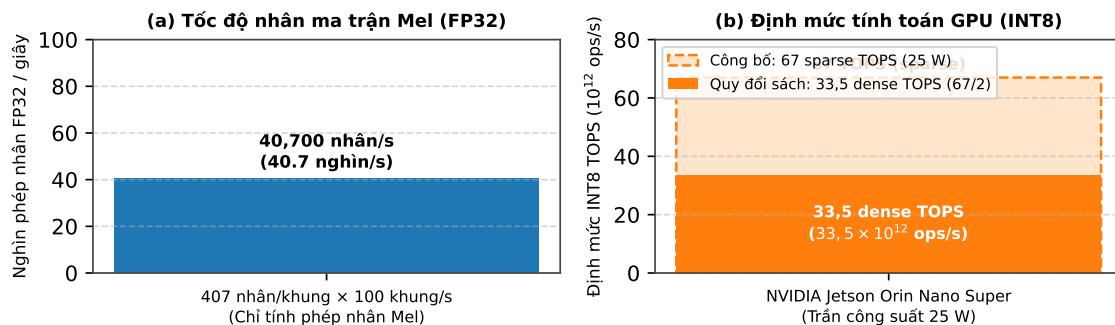
Đối chiếu quy mô giữa thông lượng tính toán của GPU và tốc độ nhân ma trận Mel:

$$\text{quotient_mac_as_one} = \frac{33,5 \times 10^{12}}{40700} \approx 823095823,10$$

$$\text{quotient_mac_as_two} = \frac{33,5 \times 10^{12}}{2 \times 40700} \approx 411547911,55$$

Con số 407 trong nhật ký kiểm thử là số phép nhân mỗi khung (61 hàng đa điểm thực tế cần 346 phép cộng). Thương số thứ nhất xem mỗi phép nhân tích lũy (MAC) là một thao tác (40.700 thao tác/giây). Thương số thứ hai dựa trên quy ước thiết kế đếm một phép MAC gồm hai thao tác (1 nhân + 1 cộng, tức $2 \times 40.700 = 81.400$ thao tác/giây). Dù tính theo quy ước nào, tỷ lệ quy mô vẫn đứng ở bậc hàng trăm triệu lần (823095823,10 và 411547911,55), và lập luận so sánh hoàn toàn không phụ thuộc vào việc chọn thương số nào.

Đối chiếu quy mô: Khác biệt bậc độ lớn $\sim 10^8$ lần (33,5 TOPS vs 40.700 FP32 nhân/s)



Hình 1.4d: **Bản chất:** Đối chiếu quy mô giữa tốc độ nhân ma trận Mel (FP32) và định mức tính toán công bố của Edge GPU (INT8). **Điểm cần quan sát:** Khối lượng 40.700 phép nhân/giây chỉ tính riêng cho ma trận Mel; định mức công bố của GPU là 67 sparse INT8 TOPS (quy đổi 33,5 dense INT8 TOPS); hai trục có thứ nguyên hoàn toàn khác nhau; thương số quy mô đạt 823095823,10 (nếu tính 1 MAC = 1 op) và 411547911,55 (nếu tính 1 MAC = 2 ops). **Ý nghĩa:** Tải tính toán số học của phép nhân Mel hoàn toàn chìm ngấm trước năng lực tính toán cực đại của GPU, cho thấy phép so sánh thông lượng thuần túy không phải là căn cứ để chuyển đổi sang FPGA.

Tài liệu AMD DS890 công bố thiết bị XCK26 tích hợp 1248 lát cắt DSP48E2. Tuy nhiên, khối lượng số học tiền xử lý nhỏ bé này không được dùng để suy diễn ra một tỉ lệ phần trăm tải chip hay tần số xung nhịp.

★ PHÁT HIỆN THEN CHỐT

Khối tiền xử lý âm thanh quá nhỏ bé khi đặt cạnh định mức công bố của GPU, do đó một phép so sánh thuần túy về thông lượng tính toán không thể là lý do thuyết phục để biện minh cho việc chuyển đổi kiến trúc sang FPGA.

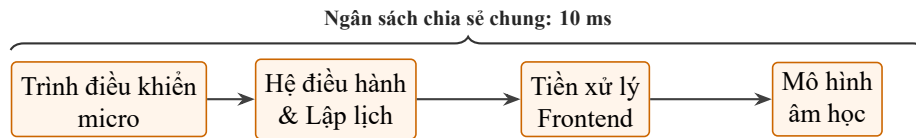
5. Cơ Chế Hàng Đợi: Nguyên Nhân Gây Nguy Cơ Trễ Hạn Chót 10 ms

Câu hỏi giai đoạn trước không thể trả lời: Nếu khối lượng tính toán số học nhỏ đến mức chìm ngấm như vậy, điều gì vẫn có thể khiến hệ thống vi phạm hạn chót 10 ms của bước nhảy?

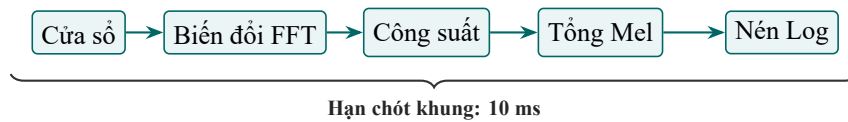
Xuất phát từ quan sát về tốc độ tính toán ở Cảnh 4, hai giả thuyết kiến trúc được đặt ra:

- **Giả thuyết A (Đường truyền Edge GPU):** Ngân sách thời gian 10 ms là một hàng đợi tài nguyên chia sẻ chung giữa trình điều khiển micro, hệ điều hành (lập lịch phân phối thời gian CPU/GPU), luồng tiền xử lý frontend, và quá trình suy luận của mô hình âm học. Dù khối lượng số học rất nhỏ, hạn chót 10 ms vẫn có nguy cơ bị vi phạm nếu hàng đợi chia sẻ này phát sinh độ trễ phụ (overhead), nghẽn luồng truyền dữ liệu hoặc bị ngắt quãng bởi các tác vụ hệ thống.
- **Giả thuyết B (Đường truyền FPGA):** Các toán tử tiền xử lý tương tự được bố trí trực tiếp trên các đường dây vật lý độc lập (dedicated wires) tách biệt hoàn toàn khỏi hàng đợi phần mềm. Hạn chót 10 ms chỉ bị vi phạm nếu bản thân chuỗi mắt xích trên các đường dây vật lý đó tốn nhiều hơn 10 ms để hoàn tất.

(a) Đường truyền Edge GPU: Chia sẻ chung một tiến trình hàng đợi



(b) Đường truyền FPGA: Các toán tử đặt trên đường dây vật lý độc lập



Hình 1.4e: **Bản chất:** So sánh mô hình hàng đợi chia sẻ tài nguyên (Edge GPU) và mô hình đường truyền dữ liệu chuyên dụng (FPGA). **Điểm cần quan sát:** Trên Edge GPU, toàn bộ trình điều khiển, hệ điều hành, tiền xử lý và mô hình cùng tranh chấp hạn ngạch thời gian 10 ms; trên FPGA, các toán tử được kết nối bằng đường dây vật lý độc lập. **Ý nghĩa:** Sự khác biệt kiến trúc nằm ở cơ chế quản lý hàng đợi và tính độc lập của luồng dữ liệu chứ không nằm ở việc rút ngắn thời gian tính toán của từng phép nhân.

► NHÌN THẤY VẬT LÝ

Khối lượng số học không quyết định việc bảo toàn hạn chót; việc chia sẻ tài nguyên trong hàng đợi có thể gây trễ; chương này chưa có số đo trên mạch để chốt lại kết luận đó.

6. Ranh Giới Một Giấy Âm Thanh và Chi Phí Độ Trễ Gom Cụm

Câu hỏi giai đoạn trước không thể trả lời: Điều gì còn lại ở ranh giới kết thúc 1 giây âm thanh, và kỹ thuật gom cụm khung (batching) tác động như thế nào đến độ trễ hệ thống?

Xét một đoạn tín hiệu âm thanh có thời lượng chuẩn 1 giây, gồm 16.000 mẫu tại $f_s = 16.000$ Hz. Điểm bắt đầu hợp lệ cuối cùng của một khung $L = 400$ mẫu là bội số lớn nhất của bước nhảy $H = 160$ mẫu thỏa mãn điều kiện không vượt quá giới hạn $16000 - 400 = 15600$:

$$97 \times 160 = 15520$$

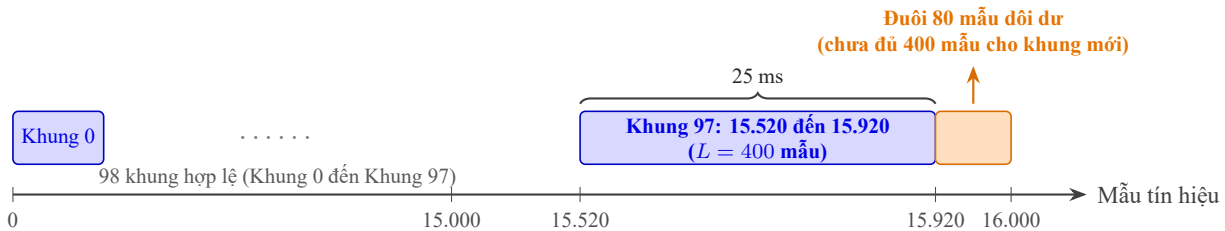
Do đó, các khung hợp lệ chạy từ Khung 0 đến Khung 97, tạo thành tổng cộng `frames_in_one_second = 98` khung âm thanh. Khung cuối cùng (Khung 97) bắt đầu tại mẫu 15520 và kết thúc tại:

$$15520 + 400 = 15920$$

Phần đuôi dữ liệu còn lại ở cuối giây âm thanh gồm:

$$16000 - 15920 = 80 \text{ mẫu}$$

Phần 80 mẫu dôi dư này không bao giờ đủ 400 mẫu để cấu thành một khung hợp lệ mới nếu không có các biện pháp chèn số 0 giả định.



batch = 1 là lựa chọn độ trễ. Gom cụm $B = 8$ khung tích lũy thêm $(8 - 1) \times 10 \text{ ms} = 70 \text{ ms}$ trước khi khung đầu được xử lý.

Hình 1.4f: **Bản chất:** Ranh giới phân khung ở cuối 1 giây âm thanh và phần mẫu dôi dư không đủ điều kiện tạo khung. **Điểm cần quan sát:** Khung thứ 98 (Khung 97) kết thúc tại mẫu 15.920; 80 mẫu cuối cùng từ 15.920 đến 16.000 bị bỏ sót vì không đủ chiều dài 400 mẫu; việc gom lô $B = 8$ phát sinh thêm 70 ms chờ đợi. **Ý nghĩa:** batch = 1 là lựa chọn đánh đổi độ trễ: gom thêm mỗi khung làm trễ thêm 10 ms; một lô $B = 8$ khung tích lũy thêm 70 ms trước khi khung đầu tiên được đưa vào xử lý.

Trong kiến trúc xử lý luồng thời gian thực, việc chờ đợi tích lũy B khung để gom thành một lô (batch) sẽ làm gia tăng thêm một khoảng trễ thời gian vật lý:

$$\Delta t_{\text{batch}} = (B - 1) \times 10 \text{ ms}$$

Cụ thể, một cấu hình gom lô kích thước $B = 8$ khung làm trễ thêm $(8 - 1) \times 10 \text{ ms} = 70 \text{ ms}$ tích lũy trước khi khung đầu tiên được chuyển đến bộ xử lý. batch = 1 là lựa chọn độ trễ để tránh khoảng thời gian chờ tích lũy hàng đợi này.

Câu Nối Sang Mục 1.5: Ba Câu Hỏi Mở Về Ranh Giới Phần Cứng

Phân tích kiến trúc trong mục này để lại ba câu hỏi mở cần giải quyết bằng thực nghiệm đo đạc:

1. Điều gì sẽ xảy ra với độ phân giải tần số và dung lượng bộ đệm nếu chiều dài khung cửa sổ bị cắt giảm một nửa ($L = 200$ mẫu thay vì 400 mẫu)?
2. Điều gì sẽ xảy ra với băng thông dữ liệu và kích thước FFT nếu tín hiệu âm thanh đầu vào được lấy mẫu ở tần số chuẩn 48 kHz thay vì 16 kHz?
3. Điều gì sẽ xảy ra với hiệu năng tính toán và diện tích mạch nếu hệ thống giữ nguyên bảng trọng số ma trận dày 80×257 thay vì lược bỏ và chỉ lưu các trọng số thực dùng?

Toàn bộ các giá trị tính toán và dữ liệu đo đạc thực nghiệm trong mục này được trích xuất từ nhật ký kiểm thử `results/lab14/20261005T230515Z/stdout.log` (với mã băm toàn vẹn được lưu giữ trong tập tin kiểm soát `results/SHA256SUMS`), bảng thông số kỹ thuật kiến trúc AMD/Xilinx DS890 v4.10 (Bảng 23, dòng thiết bị XCK26), và tài liệu định mức công bố của NVIDIA Jetson Orin Nano Super.